COLLEGE MANAGEMENT SYSTEM USING MONGODB

1. INTRODUCTION

The College Management System is designed to manage and organize data related to students, staff, departments, and library books efficiently. This system uses MongoDB as a NoSQL database to store and manage data in a flexible and scalable way.

2. OBJECTIVE

- To manage student, staff, department, and book records
- To implement database relationships using MongoDB
- To perform basic CRUD operations
- To demonstrate a real-world database application

3. TECHNOLOGIES USED

Backend: Node.js, Express.js
Database: MongoDB
ODM: Mongoose

4. SYSTEM MODULES

1. Student Management
2. Staff Management
3. Department Management
4. Library (Books) Management

5. DATABASE DESIGN

Collections:
- Department
- Student
- Staff
- Book

Relationships:
- Student → references Department
- Staff → references Department

6. SCHEMA DESIGN

Department Schema:
const departmentSchema = new mongoose.Schema({ name: String, hod: String });

Student Schema:
const studentSchema = new mongoose.Schema({ name: String, age: Number, department: { type: mongoose.Schema.Types.ObjectId, ref: 'Department' } });

Staff Schema:
const staffSchema = new mongoose.Schema({ name: String, role: String, department: { type: mongoose.Schema.Types.ObjectId, ref: 'Department' } });

Book Schema:
const bookSchema = new mongoose.Schema({ title: String, author: String, available: Boolean });

7. IMPLEMENTATION

MongoDB Connection:
mongoose.connect('mongodb://127.0.0.1:27017/collegeDB');

Add Student API:
app.post('/addStudent', async (req, res) => { const student = new Student(req.body); await student.save(); res.send(student); });

Get Students API:
app.get('/students', async (req, res) => { const students = await Student.find().populate('department'); res.send(students); });

8. SAMPLE DATA

Department:
{ name: 'Computer Science', hod: 'Dr. Rao' }

Student:
{ name: 'Uday', age: 20, department: 'OBJECT_ID' }

9. OUTPUT

Successfully added and retrieved student data.
Relationship between student and department displayed.

10. ADVANTAGES

- Flexible schema (NoSQL)
- Scalable and fast
- Easy to modify structure

11. FUTURE ENHANCEMENTS

- Add authentication system
- Implement frontend UI
- Add attendance system
- Book issuing and return system

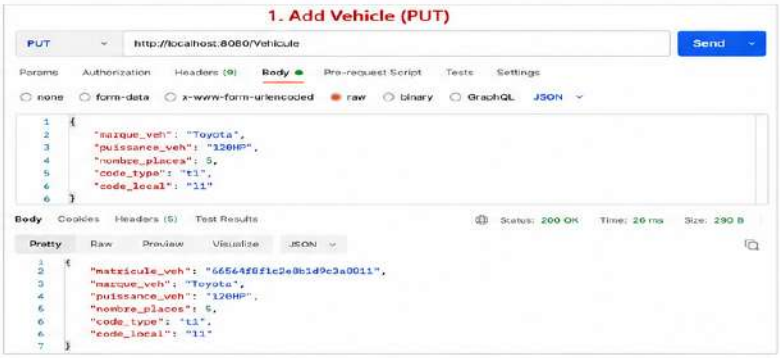
12. CONCLUSION

The College Management System using MongoDB provides an efficient way to manage college data. It demonstrates the use of NoSQL databases and relationships using ObjectId references.

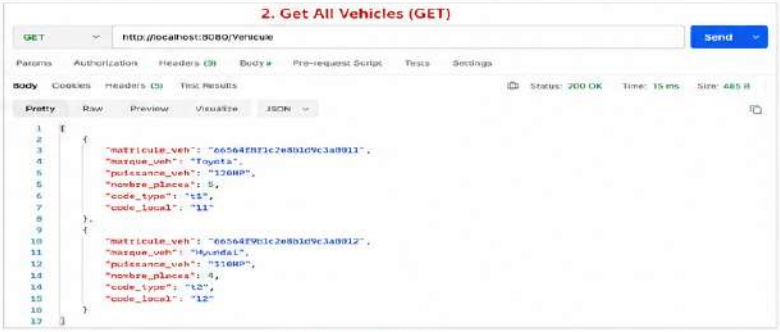
13. REFERENCES

- MongoDB Documentation
- Node.js Documentation
- Express.js Documentation

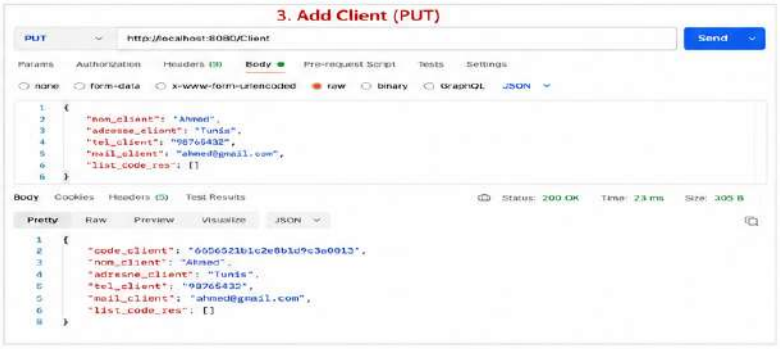
1. Add Vehicle (PUT)



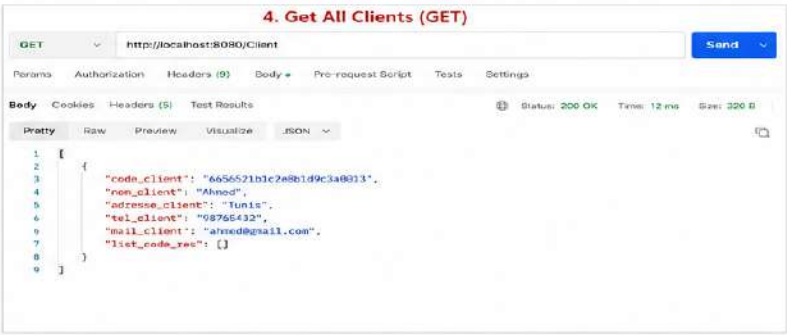
2. Get All Vehicles (GET)



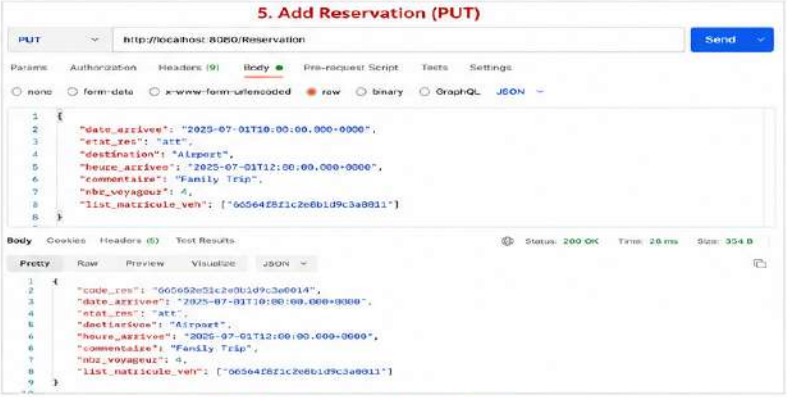
3. Add Client (PUT)



4. Get All Clients (GET)



5. Add Reservation (PUT)



6. Get Pending Reservations (GET /att)